

PARALLEL IMPLEMENTATION OF A PRESSURE-CORRECTION PROJECTION METHOD FOR THE UNSTEADY INCOMPRESSIBLE NAVIER-STOKES EQUATIONS ^{*1)}

Ju Li-li Zhang Lin-bo

*(State Key Laboratory of Scientific/Engineering Computing,
Institute of Computational Mathematics and Scientific/Engineering Computing,
Chinese Academy of Sciences, Beijing)*

Abstract

This paper deals with parallel implementation on distributed memory systems of a pressure-correction projection scheme for the unsteady incompressible Navier-Stokes equations, the CNMT2 scheme (i.e., the Crank-Nicolson Modified Temam scheme II), presented in [1, 3]. The key point of this work is to study parallelization of the fast Poisson solver [3] and analyse its parallel efficiency. Various techniques, such as pipelining and canon cyclic algorithm, were used to ensure good parallel performance and scalability of the algorithm. The algorithm has been implemented using MPI message passing environment and numerical tests have been carried out on various computers, including the home made Dawn-1000 MPP system and workstation clusters.

1. Introduction

Consider the unsteady incompressible Navier-Stokes equations (INSE) for incompressible flows:

$$\frac{\partial \mathbf{w}}{\partial t} + u \frac{\partial \mathbf{w}}{\partial x} + v \frac{\partial \mathbf{w}}{\partial y} + \text{grad } p = \frac{1}{Re} \text{div grad } \mathbf{w} + \mathbf{b}(x, y, t) \quad (1)$$

$$\text{div } \mathbf{w} = 0 \quad (2)$$

* Received November 17, 1997.

¹⁾ Supported by National Natural Sciences Fundation (19772056) and 863 Projects.

©1998 by Allerton Press, Inc. Authorization to photocopy individual items for internal or personal use, or the internal or personal use of specific clients, is granted by Allerton Press, Inc. for libraries and other users registered with the Copyright Clearance Center (CCC) Transactional Reporting Service, provided that the base fee of \$50.00 per copy is paid directly to CCC, 222 Rosewood Drive, Danvers, MA 01923.

in a two-dimensional region Ω , where $\mathbf{w}$ represents the velocity and p the pressure. For simplicity, we consider the case of Dirichlet boundary condition, i. e., the value of $\mathbf{w}$ on $\partial\Omega$ is known for all time t which satisfies the following constraint:

$$\oint_{\partial\Omega} \mathbf{w} \cdot \mathbf{n} \, ds = 0 \quad (3)$$

The initial value of $\mathbf{w}$, $\mathbf{w}(x, y, 0)$, is given and also satisfies constraint (3).

The main difficulty in the solution of this problem is that the system is not entirely evolutionary, it is subject to the divergence-free constraint (2). The pressure correction (PC) projection method proposed by van Kan [5] has been proved to be very effective for this problem. The key point in the pressure correction schemes is efficient solution of resulting Poisson equation for the pressure.

In this papers, we discuss parallel implementation issues of a pressure correction projection type scheme on semi-staggered grids, the CNMT2 scheme. This scheme was introduced in [1] and a fast pressure Poisson equation solver for it was presented in [3]. The CNMT2 scheme and the fast Poisson equation solver have been shown to be quite efficient for solving unsteady incompressible Navier-Stokes equations. We mainly focus on the parallel implementation of the fast Poisson solver on distributed memory parallel systems and analyze its parallel efficiency.

2. The CNMT2 Scheme and the Fast Solver for Pressure Poisson Equation

In this section, we briefly introduce the CNMT2 scheme and the fast solver for the resulting pressure Poisson equations, which were presented in details in [1, 3].

After spacial discretization by finite difference, equations (1) and (2) can be written in the following form:

$$\frac{\partial \mathbf{w}}{\partial t} = \mathbf{F}(\mathbf{w}, t) - G \, p \quad (1)$$

$$D \, \mathbf{w} = 0 \quad (2)$$

where $\mathbf{F}(\mathbf{w}, t)$ includes the discretization of convection, diffusion and source terms and the boundary condition, G and D represent respectively the discretization of *grad* and *div* operators. The CNMT scheme on half-staggered grids is written as:

$$\frac{\tilde{\mathbf{w}} - \mathbf{w}^n}{\Delta t / 2} + \mathbf{F}(\tilde{\mathbf{w}}, t_{n+\frac{1}{2}}) + G \, p^n = 0 \quad (3)$$

$$\frac{\bar{\mathbf{w}} - \tilde{\mathbf{w}}}{\Delta t / 2} + \frac{1}{2} \phi = 0 \quad (4)$$

$$DG \, \phi = \frac{D \, \tilde{\mathbf{w}}}{\Delta t / 4} \quad (5)$$

$$p^{n+1} - p^n = \phi \quad (6)$$

where $t_n = n\Delta t$, Δt is the time step, $p^n = p|_{t=t_n}$, $w^n = w|_{t=t_n}$, $\bar{w} = \frac{w^{n+1} + w^n}{2}$, $\bar{w}$ is the auxiliary velocity field. Assuming w^n is known, then w^{n+1} is computed by the following steps:

Algorithm 1

1. Given w^n , find $\bar{w}$ by (3).
2. solve the discrete Poisson equation (5) for ϕ .
3. compute w^{n+1} and p^{n+1} by Equations (4) and (6).

We assume that a non uniform $m \times n$ rectangular grid is used, (x_i, y_j) denotes the coordinates of grid points ($i = 1, \dots, m, j = 1, \dots, n$). Let $dx_i = x_{i+1} - x_i$, $dy_j = y_{j+1} - y_j$, $dx_{i,i+1} = (dx_{i+1} + dx_i)/2$, $dy_{j,j+1} = (dy_{j+1} + dy_j)/2$.

In the CNMT2 scheme, the convection terms in F in equation (3) is written as the average of conservative and nonconservative form:

$$\begin{cases} u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} = \frac{1}{2} \left(u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} \right) + \frac{1}{2} \left(\frac{\partial uu}{\partial x} + \frac{\partial uv}{\partial y} \right) \\ u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} = \frac{1}{2} \left(u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} \right) + \frac{1}{2} \left(\frac{\partial vu}{\partial x} + \frac{\partial vv}{\partial y} \right) \end{cases} \quad (7)$$

using the condition $\text{div } w = 0$. The auxiliary velocity field $\bar{w}$ in equation (3) is solved by ADI method with at most four iterations using a linearized form of F , so this step consists of solution of a set of tridiagonal linear systems which requires $O(mn)$ operations. The resulting scheme is unconditionally stable, and is called CNMT2+PC scheme, see [1, 7]. In order to avoid deviation of the solution, p^{n+1} may be updated again by solving the following so called *component consistency equation*:

$$DGp^{n+1} = DF(w^{n+1}, t_{n+1}) \quad (8)$$

(see [1]), and the corresponding method is called CNMT2+PCC method.

In [2], a new component-consistent pressure correction projection scheme was proposed which avoids the deviation problem without the need of the solution of Equation (8). Although we discuss the parallelization procedure with the CNMT2+PCC scheme, the method applies directly to the new scheme.

Let $L = DG$, $R = \frac{D\bar{w}}{\Delta t/4}$, then Equation (5) becomes $L\phi = R$. The matrix L can be decomposed into the following form:

$$L = [C_n \otimes D_x A_m + D_y B_n \otimes C_m] \quad (9)$$

where

$$C_l = \begin{bmatrix} 1 & 1 & & & \\ 1 & 2 & 1 & & \\ & \ddots & & \ddots & \\ & & & 1 & 2 & 1 \\ & & & & 1 & 1 \end{bmatrix}_{l \times l}$$

$$D_x = \text{diag} \left[\frac{1}{dx_1}, \dots, \frac{1}{dx_m} \right], \quad D_y = \text{diag} \left[\frac{1}{dy_1}, \dots, \frac{1}{dy_n} \right]$$

A_m and B_n are two tridagonal matrices depending on dx_i and dy_j respectively. A_m is an $m \times m$ matrix, B_n an $n \times n$ matrix.

According to QZ algorithm and properties of generalized eigenvalues, there exists eigenvalues (α_i, β_i) and eigenvectors v_i ($i = 1, 2, \dots, m$) such that:

$$\beta_i D_x A_m v_i = \alpha_i C_m v_i$$

Let D_a, D_b be diagonal matrices whose diagonal elements are α_i, β_i respectively. We have

$$D_x A_m V D_b = C_m V D_a$$

where $V = [v_1, v_2, \dots, v_m]$. The matrices A_m and C_m both have one zero eigenvalue, so do the matrices D_a and D_b . We can order the eigenvectors such that:

$$\alpha_m = 0, \quad \beta_1 = 0$$

Let

$$U = [D_x A_m v_1, D_x A_m v_2, \dots, D_x A_m v_{m-1}, C_m v_m]$$

then it is not difficult to see that

$$U^{-1} (D_x A_m) V = \text{diag} [1, \dots, 1, 0] = \bar{I}_m$$

$$U^{-1} C_m V = \text{diag} [0, \beta_2/\alpha_2, \dots, \beta_{m-1}/\alpha_{m-1}, 1] = D_x$$

Thus, the tridiagonalization of Equation (1) can be achieved as follows:

$$\begin{aligned} (I_n \otimes U^{-1}) L (I_n \otimes V) &= C_n \otimes U^{-1} D_x A_m V + D_y B_n \otimes U^{-1} C_m V \\ &= C_n \otimes \bar{I}_m + D_y B_n \otimes D_x \\ &\stackrel{\text{def}}{=} \bar{L} \end{aligned}$$

$\bar{L}$ has three non-zero diagonals with bandwidth m , and Equation (5) becomes

$$\bar{L} \bar{\Phi} = \bar{R} \quad (10)$$

where $\bar{\Phi} = (I_n \otimes V^{-1}) \Phi$, $\bar{R} = (I_n \otimes U^{-1}) R$.

We start with an ordering of the grid points first in x -direction, then in y -direction. Let P be the permutation matrix which reorders the grid points first in y -direction then in x -direction, we get

$$\bar{L} \bar{\Phi} = \bar{R} \quad (11)$$

here $\bar{L} = P \bar{L} P$, $\bar{\Phi} = P \bar{\Phi}$, $\bar{R} = P \bar{R}$, and $\bar{L}$ is a block diagonal matrix, its diagonal elements T_i are $n \times n$ tridiagonal matrices:

$$\begin{aligned} T_1 &= C_n \\ T_i &= C_n + \frac{\beta_i}{\alpha_i} D_y B_n \quad (i = 2, 3, \dots, m-1) \\ T_m &= D_y B_n \end{aligned} \quad (12)$$

The fast solver for the pressure Poisson equation can be summarized as follows:

Algorithm 2

1. *Preprocess. Compute generalized eigen-decomposition (D_a, D_b, V) of $(D_x A_m, C_m)$, then compute U and T_i . This needs to be done only once before the first time step and it costs $O(m^3)$ operations.*
2. *Solve $(I \otimes U) \tilde{R} = R$. The cost is $O(nm^2)$ operations.*
3. *Solve $\bar{L} \bar{\Phi} = \bar{R} = P \tilde{R}$. The cost of this step is $O(nm)$ operations.*
4. *Compute $\Phi = (I \otimes V) P \bar{\Phi}$. It costs $O(nm^2)$ operations.*

See [3].

3. Parallel Algorithm

Suppose we have p processors, $p = p_x \times p_y$, p_x and p_y are the number of processors allocated for x -direction and y -direction respectively. The computational domain is partitioned according to processor topology: the grid points $(x_{r,s}, y_{r,s})$, $m_J \leq r < m_{J+1}$ and $n_K \leq s < n_{K+1}$, are allocated to processor $P_{J,K}$ ($0 \leq J \leq p_x - 1$, $0 \leq K \leq p_y - 1$), where $m_0 = 1$, $m_{J+1} - m_J = nx_J$, $m_{p_x} = m + 1$; $n_0 = 1$, $n_{K+1} - n_K = ny_K$, $n_{p_y} = n + 1$.

Due to the locality nature of the calculations, each processor can perform the updates on interior grid points in parallel. In order to perform the updates on inner boundary grid points, neighbouring processors exchange informations with each other in a usual way. We omit the details here.

When computing the auxiliary velocity field using ADI method (Step 1 of Algorithm 1), we have to solve a set tridiagonal linear systems alternatively in each spacial direction. This problem is generally considered a difficult one to obtain good parallel efficiency on distributed memory systems, and is very common in computational fluid dynamics. We solve this problem with a method presented in [4], i.e., by organizing the computations in a pipelined fashion using a parameter which controls the granularity of the computation. Numerical experiences show that by properly choosing the granularity parameter, pipelined method can achieve excellent parallel efficiency.

The most important part of the parallel solver is the implementation of fast pressure Poisson equation solver presented in last section, which is the most time-consuming part of the whole algorithm. Since the generalized eigen-decomposition only needs to be done once before the first time step, and the order of the matrices is not very large (m), we use an existing sequential program to perform it. The third step of Algorithm 2 consists of simultaneous solution of a set of tridiagonal linear systems, which is implemented using pipelining as for the ADI method. The second and fourth steps of the fast solver are in fact matrix multiplication problems, but with a special structure and distribution of submatrices, since they represents the most computation and communication intensive part of the fast solver, we will describe the implementation in little more details.

The second step of Algorithm 2 consists of computation of $\tilde{R} = (I_n \otimes U^{-1})R$. Since U is a constant matrix, we compute U^{-1} and store it before the first time step, thus the second and fourth steps of Algorithm 2 perform the same kind of matrix multiplication.

Consider the computation of $\Phi = (I_n \otimes V)\tilde{\Phi}$. Denote by $\tilde{\Phi}_{J,K}$ (respectively $\Phi_{J,K}$) the vector composed of components of $\tilde{\Phi}$ (respectively Φ) allocated to processor $P_{J,K}$. Let

$$\tilde{\Phi}_K = (\tilde{\Phi}_{0,K}, \tilde{\Phi}_{1,K}, \dots, \tilde{\Phi}_{p_x-1,K})$$

Then we have:

$$\begin{aligned} (I_n \otimes V)\tilde{\Phi} &= \begin{bmatrix} I_{ny_0} \otimes V & & & \\ & I_{ny_1} \otimes V & & \\ & & \ddots & \\ & & & I_{ny_{p_y-1}} \otimes V \end{bmatrix} \begin{pmatrix} \tilde{\Phi}_0 \\ \tilde{\Phi}_1 \\ \vdots \\ \tilde{\Phi}_{p_y-1} \end{pmatrix} \\ &= \begin{bmatrix} (I_{ny_0} \otimes V)\tilde{\Phi}_0 & & & \\ & (I_{ny_1} \otimes V)\tilde{\Phi}_1 & & \\ & & \ddots & \\ & & & (I_{ny_{p_y-1}} \otimes V)\tilde{\Phi}_{p_y-1} \end{bmatrix} \end{aligned}$$

It is easy to see that:

$$\Phi_{J,K} = \sum_{I=0}^{p_x-1} (I_{ny_K} \otimes V_{J,I})\tilde{\Phi}_{I,K}, \quad 0 \leq J \leq p_x - 1, \quad 0 \leq K \leq p_y - 1 \quad (1)$$

where

$$V = \begin{bmatrix} V_{0,0} & V_{0,1} & \cdots & V_{0,p_x-1} \\ V_{1,0} & V_{1,1} & \cdots & V_{1,p_x-1} \\ \vdots & \vdots & \ddots & \vdots \\ V_{p_x-1,0} & V_{p_x-1,1} & \cdots & V_{p_x-1,p_x-1} \end{bmatrix}$$

$V_{i,j}$ is an $n_{x_i} \times n_{x_j}$ matrix. So the computation on processor $P_{J,K}$ is only related with the processors $\{P_{i,K}, i = 0, \dots, p_x - 1\}$, we use the canon cyclic algorithm to compute $\Phi_{J,K}$ as follows:

Algorithm 3 Computation of $\Phi_{J,K}$ on processor $P_{J,K}$:

$\Phi_{J,K} = 0$

$\Phi_{\text{temp}} = \tilde{\Phi}_{J,K}$

do $i = 0, p_x - 1$

!!!!!!!!!!!!!!!!!!!!!!!!!!!!

if (i.gt.0) then recv Φ_{temp} from processor $\text{mod}(J-1+p_x, p_x)$

if (i.lt.(p_x-1)) then send Φ_{temp} to processor $\text{mod}(J+1, p_x)$

should be:

recv Φ_{temp} from processor $(J+p_x-1 \bmod p_x, K)$

```

send  $\Phi_{\text{temp}}$  to processor  $(J+1 \bmod p_x, K)$ 
compute  $\Phi_{J,K} = \Phi_{J,K} + (I_{n \times J} \otimes V_{J,i}) \Phi_{\text{temp}}$ 
enddo

```

For Algorithm 3 to work efficiently, asynchronous communications are needed to overlap the computations and communications. The total amount of data sent and received per processor is $O(mn(p_x - 1)/p)$, and the number of floating point operations is $O(nm^2/p)$, when the problem size compared with the number of processors is sufficiently large or the communication is sufficiently fast, the communications will be masked by calculations, and only the startup time in the communications (latency) affects total computational time and parallel efficiency.

Let T_0 be the time required by one floating point operation, T_1 the time required for communication of one floating point number, T_2 the startup time of one communication. Then the total calculation time of the above algorithm is:

$$T_{\text{calc}} = c_0 T_0 n m^2 / p$$

The total communication time is:

$$T_{\text{comm}} = c_1 T_1 m n (p_x - 1) / p$$

The total startup time is:

$$T_{\text{lat}} = T_2 (p_x - 1)$$

c_0 and c_1 are constants. If communications and calculations are done simultaneously, then the total computational time is:

$$T = \max(T_{\text{calc}}, T_{\text{comm}}) + T_{\text{lat}} = \max(c_0 T_0 n m^2 / p, c_1 T_1 m n (p_x - 1) / p) + T_2 (p_x - 1) \quad (2)$$

and the parallel efficiency is

$$E = \left(1 + \frac{T_2 p (p_x - 1)}{\max(c_0 T_0 n m^2, c_1 T_1 m n (p_x - 1))} \right)^{-1} \quad (3)$$

The matrix multiplication on each processors is performed by calling the level 3 BLAS routine DGEMM (matrix-matrix multiplication) [9] while in in [1, 3] it was done by calling the level 2 BLAS routine DGEMV (matrix-vector multiplication) [8]. It turns out that replacing DGEMV by DGEMM greatly improves overall performance of the program.

Finally, we summarize the parallel version of CNMT2+PCC algorithm as follows:

Algorithm 4

1. *Preprocess.* Each processor independently computes (D_a, D_b, V) and U, U^{-1}, T_i . The complexity of this step is $O(m^3)$ but it needs to be done only once before the first time step.
2. *Solve Equation (8) using Algorithm 3.* The computational complexity is $O(nm^2/p)$.

3. Use pipelined ADI method and Equation (3) to compute the auxiliary velocity field $\tilde{\mathbf{w}}$. The computational complexity is $O(nm/p)$.
4. Use Algorithm 3 and Equation (5) to compute Φ . Then update the velocity and the pressure using Equations (4) and (6), and advance to next time step. The computational complexity of this step is $O(nm^2/p)$.
5. Goto Step 2.

Algorithm 4 is called PCNMT2+PCC algorithm.

4. Numerical Experiments and Analysis

We have implemented the PCNMT2+PCC algorithm in Fortran with MPI as the message passing interface. To test its correctness and efficiency, we have used it to compute the two-dimensional driven cavity flow on some parallel systems. The results obtained with PCNMT2+PCC scheme were identical with those obtained by CNMT2+PCC scheme in [6].

Since we are mainly concerned with the parallel performance of our algorithm, we will not give details of numerical results.

Table 1–6 give the execution time of PCNMT2+PCC algorithm on Dawn 1000 and a workstation cluster (SGI INDIGO, 100Mbps ATM network). In the tables, P_x and P_y represent the numbers of processors in x -direction and y -direction respectively, Grid Points is average number of grid points in each subdomain, l_x and l_y are the pipeline lengths when solving tridigonal systems in x -direction and y -direction respectively, the values used are estimated optimal ones according to the parallel model presented in [4], T_{poi} is average wall time for solving the pressure Poisson equation in each time step, T_{step} is average wall time of an entire time step. Since the performance of our program when $P_x = P_y = 1$ is higher than the code of [6], the parallel speedup and parallel efficiency are computed with respect to the time obtained for $P_x = P_y = 1$.

Table 1. Execution time on ATM workstation cluster, 197×197 grid

$P_x \times P_y$	Grid Points	l_x	l_y	$T_{\text{poi}}(s)$	$T_{\text{step}}(s)$	Parallel Speedup	Parallel Efficiency
1×1	38009	20	20	1.89	7.80	1	100.00%
1×2	19405	4	20	0.95	4.01	1.945	97.2%
2×1	19405	20	4	1.45	5.00	1.56	78.00%
1×4	9702	5	20	0.55	2.20	3.545	88.64%
2×2	9702	6	6	0.58	2.35	3.319	82.98%
4×1	9702	20	5	0.92	3.14	2.484	62.10%
1×6	9702	5	20	0.36	1.62	4.815	80.25%
2×3	9702	6	6	0.54	1.58	4.937	82.28%
3×2	9702	6	6	0.44	1.70	4.588	76.47%
6×1	9702	20	5	0.56	2.02	3.861	64.36%

Table 2. Execution time on ATM workstation cluster, 247×247 grid

$P_x \times P_y$	Grid Points	l_x	l_y	$T_{\text{poi}}(s)$	$T_{\text{step}}(s)$	Parallel Speedup	Parallel Efficiency
1×1	61009	25	25	3.66	14.13	1	100.00%
1×2	30505	5	25	1.87	7.09	1.993	99.64%
2×1	30505	25	5	2.78	8.86	1.595	79.74%
1×4	15252	7	25	1.13	3.62	3.903	97.58 %
2×2	15252	8	8	1.43	4.75	2.975	74.36 %
4×1	15252	25	7	1.71	5.41	2.612	65.30%
1×6	9702	7	25	0.66	2.70	5.233	87.22%
2×3	9702	8	9	0.85	3.11	4.543	75.72%
3×2	9702	9	8	1.07	3.53	4.002	66.71%
6×1	9702	25	7	1.32	4.37	3.233	53.89%

Table 3. Execution time on ATM workstation cluster, 297×297 grid

$P_x \times P_y$	Grid Points	l_x	l_y	$T_{\text{poi}}(s)$	$T_{\text{step}}(s)$	Parallel Speedup	Parallel Efficiency
1×1	88209	30	30	5.99	22.29	1	100.00%
1×2	44105	6	30	3.04	11.16	1.997	99.86%
2×1	44105	30	6	4.50	14.28	1.561	78.05%
1×4	22052	8	30	1.58	5.81	3.836	95.91%
2×2	22052	8	8	2.28	7.25	3.074	76.86%
4×1	22052	30	8	2.77	8.31	2.682	67.06%
1×6	9702	8	30	1.10	4.05	5.504	91.93%
2×3	9702	10	9	1.59	5.01	4.449	74.15%
3×2	9702	9	10	1.82	5.40	4.128	68.80%
6×1	9702	30	8	2.16	6.29	3.544	59.06%

Table 4. Execution time on Dawn 1000, 197×197 grid

$P_x \times P_y$	Grid Points	l_x	l_y	$T_{\text{poi}}(s)$	$T_{\text{step}}(s)$	Parallel Speedup	Parallel Efficiency
1×1	38009	20	20	1.48	25.96	1	100.00%
1×2	19405	7	20	0.78	13.13	1.977	98.86%
2×1	19405	20	7	0.95	13.49	1.924	96.22%
1×4	9702	8	20	0.46	6.91	3.757	93.92%
2×2	9702	10	10	0.51	6.94	3.741	93.52%
4×1	9702	20	8	0.75	7.60	3.416	85.39%
1×8	4851	9	20	0.30	3.98	6.523	81.53%
2×4	4851	12	12	0.32	3.80	6.832	85.39%
4×2	4851	12	12	0.44	4.06	6.394	79.93%
8×1	4851	20	9	0.78	4.96	5.234	65.42%
1×16	2426	9	20	0.25	2.75	9.440	59.00%
2×8	2426	12	16	0.23	2.29	11.336	70.85%
4×4	2426	16	16	0.28	2.31	11.238	70.24%
8×2	2426	16	12	0.48	2.82	9.206	57.54%
16×1	2426	20	9	0.95	4.12	6.301	39.38%

Table 5. Execution time on Dawn 1000, 247×247 grid

$P_x \times P_y$	Grid Points	l_x	l_y	$T_{poi}(s)$	$T_{step}(s)$	Parallel Speedup	Parallel Efficiency
1×1	61009	25	25	2.66	41.86	1	100.00%
1×2	30505	9	25	1.38	21.01	1.992	99.62%
2×1	30505	25	9	1.56	21.44	1.952	97.62%
1×4	15252	11	25	0.77	11.18	3.744	93.60%
2×2	15252	12	12	0.84	10.97	3.816	95.40%
4×1	15252	25	11	1.15	11.99	3.491	87.28%
1×8	7626	12	25	0.47	6.11	6.851	85.64%
2×4	7626	14	16	0.49	5.86	7.143	89.29%
4×2	7626	16	14	0.68	6.22	6.730	84.12%
8×1	7626	25	12	1.08	7.40	5.657	70.71%
1×16	3813	12	25	0.38	4.14	10.111	63.19%
2×8	3813	16	23	0.33	3.42	12.240	76.50%
4×4	3813	19	19	0.42	3.51	11.926	74.54%
8×2	3813	23	16	0.68	4.14	10.111	63.19%
16×1	3813	25	12	1.32	6.05	6.919	43.24%

Table 6. Execution time on Dawn 1000, 297×297 grid

$P_x \times P_y$	Grid Points	l_x	l_y	$T_{poi}(s)$	$T_{step}(s)$	Parallel Speedup	Parallel Efficiency
1×1	88209	30	30	4.34	61.82	1	100.00%
1×2	44105	10	30	2.23	31.02	1.993	99.64%
2×1	44105	30	10	3.50	33.56	1.842	92.11%
1×4	22052	13	30	1.19	15.90	3.888	97.20%
2×2	22052	14	14	1.36	16.04	3.854	96.35%
4×1	22052	30	13	1.82	17.14	3.607	90.17%
1×8	11026	14	30	0.71	8.65	7.147	89.34%
2×4	11026	18	20	0.74	8.40	7.360	91.99%
4×2	11026	20	18	1.01	8.97	6.892	86.15%
8×1	11026	30	14	1.57	10.51	5.882	73.53%
1×16	5513	15	30	0.52	5.63	10.981	68.63%
2×8	5513	20	23	0.46	4.75	13.015	81.34%
4×4	5513	23	23	0.58	4.85	12.746	79.66%
8×2	5513	23	20	0.94	5.70	10.846	67.79%
16×1	5513	30	15	1.65	7.97	7.757	48.48%

The performance of our program on Dawn 1000 are clearly better than on the workstation cluster. This is because the communication/computation ratio T_1/T_0 and communication latency T_2 on Dawn 1000 are much smaller than on the workstation cluster. Another factor which affects performance on the workstation cluster is that the tests were performed in a multi-user environment.

Figures 1 and 2 show the parallel speedup curves with respect to number of processors for varying problem sizes. In the figures the best speedup of all possible processor topologies for a given number of processors is used.

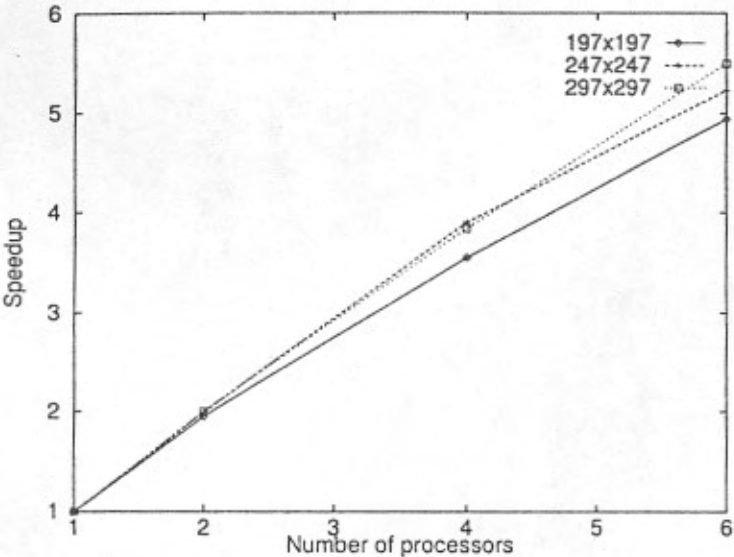


Fig. 1. Parallel speedup on ATM workstation cluster

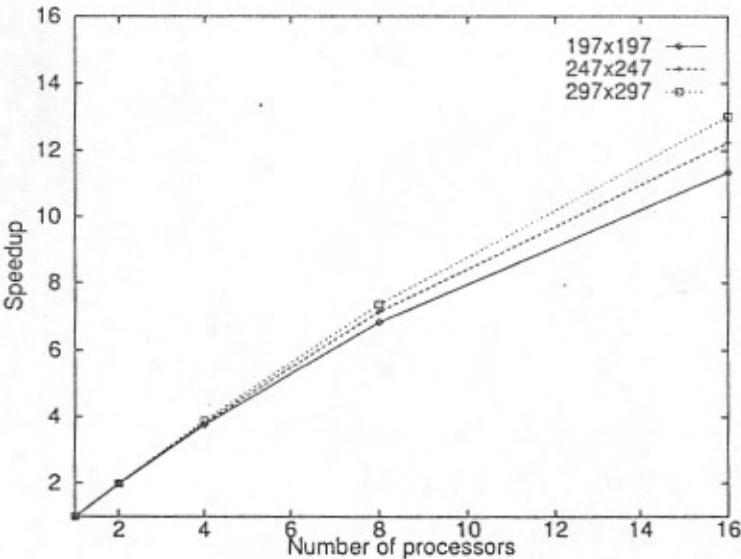


Fig. 2. Parallel speedup on Dawn 1000

5. Concluding Remarks

We have presented a parallel version of the CNMT2+PCC scheme, mainly parallel implementation of the fast pressure Poisson equation solver. Numerical

experiences presented in last section show that the PCNMT2+PCC scheme has excellent parallel efficiency and good scalability on distributed memory systems, including MPP systems and workstation clusters.

For three-dimensional problems, assume an $m \times n \times l$ grid is used, then the coefficient matrix of (9) will become

$$L = C_l \otimes C_n \otimes D_x A_m + C_l \otimes D_y B_n \otimes C_m + D_z E_l \otimes C_n \otimes C_m$$

(see [3]). The fast solver of pressure Poisson equations can be constructed similarly. Suppose the total number of processors is p , then the complexation of solving pressure Poisson equations is $O(\ln^2 m^2/p)$ in each processor, while the amount of data sent and received is $O(\ln m/(p_x p_y p_z))$ (p_x , p_y , and p_z are the number of processors in x , y , and z directions, respectively). Since the computation/communication ratio is larger than in two dimensional case, better parallel efficiency can be expected.

Acknowledgement. We would like to thank Professor Lanchieh Huang for useful discussions and advices on this problem, and provide us with the sequential code. Also we thank National Center for Intelligence Computing for giving us access to Dawn 1000 system.

References

- [1] L. C. Huang and Y. D. Wu, Highly effective and robust Scheme II for unsteady incompressible Navier-Stokes equations, *Chinese J. Comp. Math.*, **1** (1996), 58-72.
- [2] L. C. Huang and Y. D. Wu, The component-consistent pressure correction projection method for the incompressible Navier-Stokes equations, *Computer and Math. with Appl.*, **31** (1996), 1-21.
- [3] G. Golub, L. C. Huang, H. Simon, W. P. Tang, A fast solver for incompressible Navier-Stokes equations with finite difference methods, *SIAM, J. Sci. Stat. Comput.*, to appear.
- [4] Linbo Zhang, On pipelined computation of a set of recurrences on distributed memory systems, (in Chinese, English translation to appear in *Chinese J. Numer. Math. and Appl.*).
- [5] J. van Kan, A second-order accurate pressure-correction scheme for viscous incompressible flow, *SIAM, J. Sci. Stat. Comput.*, **7** (1986), 870-891.
- [6] Y. D. Wu, Highly effective and robust Pressure correction projection method for unsteady incompressible Navier-Stokes equations, Doctoral thesis of Institute of Computational Mathematics and Scientific/Engineering Computing, CAS, 1995.
- [7] N. N. Yanenko, *The method of fractional steps*, Springer-Verlag, 1981.
- [8] J. J. Dongarra, J. DuCroz, S. Hammarling, and R. Hanson, "An Extended Set of Fortran Basic Linear Algebra Subprograms", *ACM Trans. Math. Software*, **14** (1988), pp. 1-17.
- [9] J. J. Dongarra, J. DuCroz, I. S. Duff, and S. Hammarling, "A Set of Level 3 Basic Linear Algebra Subprograms: Model Implementation and Test Programs", Argonne National Laboratory Report, ANL-MCS-P88-2, 1988.